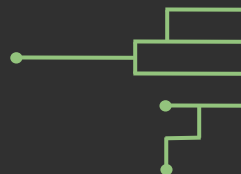


The background is dark gray with decorative circuit-like lines in light green and black in the corners. The text is centered in a white, bold, sans-serif font.

welcome to PYTHON 4 PHYSICS!

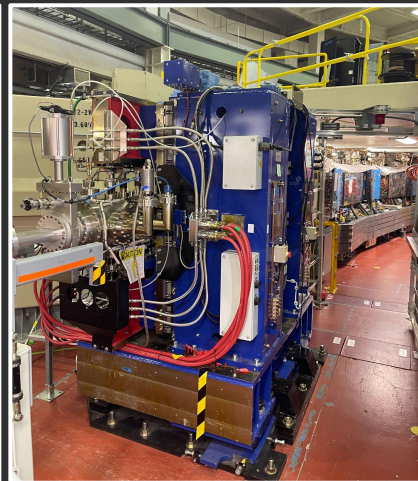
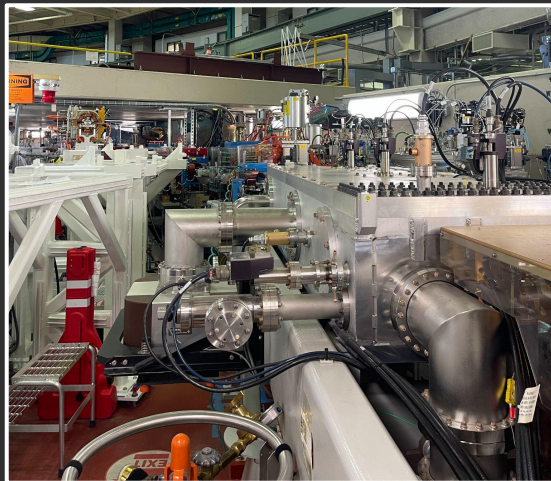
overview



- 1 Introduction
- 2 Warm-Up
- 3 Review
- 4 Data Types + Structures
- 6 Importing modules
- 7 Lecture
- 8 Control Flow
- 9 Conditionals
- 10 Operations
- 11 Instructive Example
- 12 Project
- 13

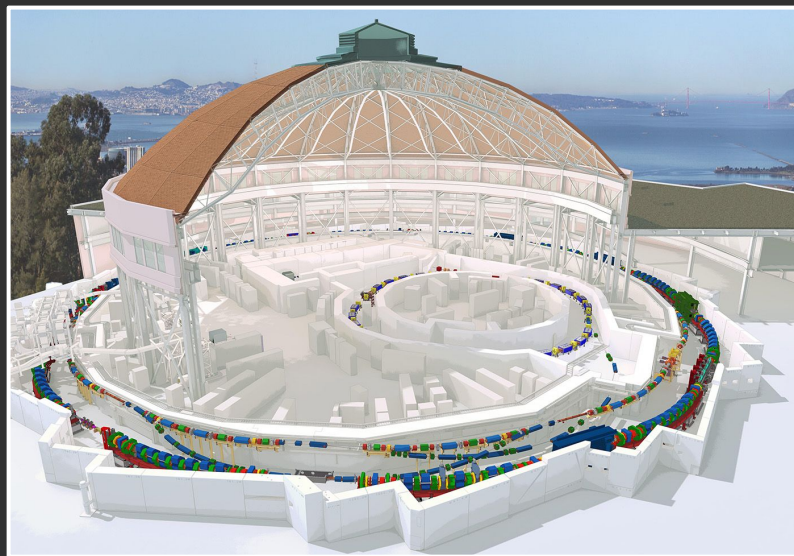
Introduction: Santiago Rodriguez

- Born in Venezuela and moved to California in 2017.
- Berkeley Physics BA '24. Transferred from a community college in San Diego.
- Currently an Accelerator Operator at the Advanced Light Source.



Python Background:

- Solar Physics Data Analysis
- Neutrino Flavor Osc. Simulations
- HelioPython Summer School
- Tutoring and Research Mentor





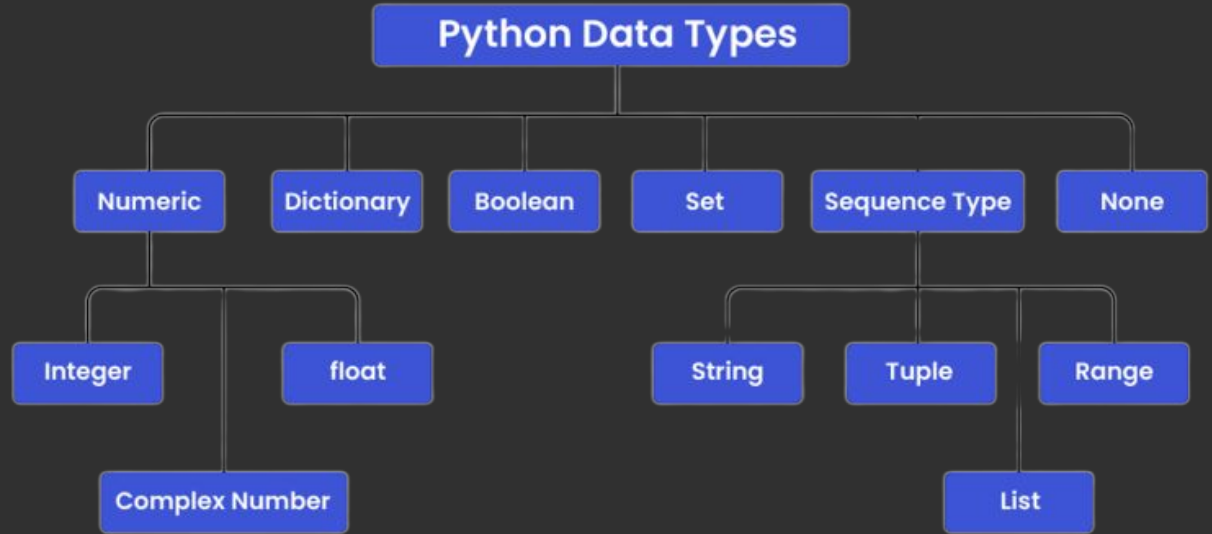
INTRO TO PYTHON

Part 2

Having talked about...

- Computer Structure > Programming Softwares > Python Interpreters.
- Syntax, Comments, and Code etiquette.
- Variables: point to objects. Python allocates memory depending on the data type.

```
<class 'NoneType'>  
<class 'int'>  
<class 'float'>  
<class 'complex'>  
<class 'bool'>  
<class 'list'>  
<class 'tuple'>  
<class 'set'>  
<class 'dict'>  
<class 'builtin_function_or_method'>  
<class 'function'>  
<class 'bytes'>  
<class 'range'>  
<class 'module'>  
<class 'frozenset'>
```



How Python Reads Code & Its Modularity

Interpreted & Procedural

Python acts like a person following a recipe. It starts at line 1 and reads downward, line-by-line.

If a variable is called before it is defined, the interpreter will stop and throw a **NameError**.



Functions and
Variables



Module



Package



Library

Python Operator Types

Operators perform mathematical, logical, and syntactic processes on your data.

Arithmetic

Math operations like addition and powers.

`+, -, *, /, %, **`

Comparison

Relational checks between values.

`==, !=, >, <, >=`

Logical

Boolean logic for condition combining.

`and, or, not`

Assignment

Assigning values to variables.

`=, +=, -=, *=, /=`

Bitwise

Operations on binary representations.

`&, |, ^, ~, <<, >>`

Membership

Testing for presence in a sequence.

`in, not in`

Identity

Checking if objects are the same.

`is, is not`

Specialized

Matrix multiplication and walrus.

`@, :=`

Control Flow & Data Handling

Conditional Logic

Directing program execution based on conditions.

- `if / elif / else`
- Tests boolean expressions in sequence.
- Indentation defines code blocks.

User Input & Output

Interacting with the user via console.

- `input()`
- Captures user data as a string.
- Use `print()` to display information back.

File Handling (I/O)

Reading from and writing to local files.

- `open()` & `with` statement
- `'r'` (read), `'w'` (write), `'a'` (append).
- Context managers ensure files close safely.

Colab & Drive Integration

Managing files in cloud environments.

- `google.colab.drive`
- `drive.mount('/content/drive')`
- Import/Export data to your Google Drive.

Importing Python Modules

Extend Python's functionality by importing external libraries for math and data visualization.

math

Standard library for basic mathematical constants and functions.

```
import math
math.sqrt(16)
math.pi
```

numpy

Powerful library for numerical computing and large multi-dimensional arrays.

```
import numpy as np
np.array([1, 2, 3])
np.mean(data)
```

matplotlib

Comprehensive library for creating static, animated, and interactive visualizations.

```
import matplotlib.pyplot as plt
plt.plot(x, y)
plt.show()
```

Use the **import** keyword to load a module. Use **as** to create an alias, which is standard practice for libraries like NumPy and Matplotlib. You can also use **from** to only call specific function from a module. For example, **from math import pi as pie**

Let's move to google colab.